

Master Thesis



Czech
Technical
University
in Prague

F3

Faculty of Electrical Engineering
Department of Cybernetics

Design of an LLM-Driven Receptionist Robot for Social Interaction

Lukáš Navara

Supervisor: doc. Mgr. Matěj Hoffmann, Ph.D.

Supervisor–specialist: MSc. Jason Khoury

Field of study: Cybernetics and Robotics

May 2026

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Navara** Jméno: **Lukáš** Osobní číslo: **502720**
Fakulta/ústav: **Fakulta elektrotechnická**
Zadávající katedra/ústav: **Katedra kybernetiky**
Studijní program: **Kybernetika a robotika**

II. ÚDAJE K DIPLOMOVÉ PRÁCI

Název diplomové práce:

Návrh recepčního robota řízeného velkým jazykovým modelem pro sociální interakci

Název diplomové práce anglicky:

Design of an LLM-Driven Receptionist Robot for Social Interaction

Jméno a pracoviště vedoucí(ho) diplomové práce:

doc. Mgr. Matěj Hoffmann, Ph.D. vidění pro roboty a autonomní systémy FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) diplomové práce:

Datum zadání diplomové práce: **04.02.2026**

Termín odevzdání diplomové práce: _____

Platnost zadání diplomové práce: **19.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Diplomant bere na vědomí, že je povinen vypracovat diplomovou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v diplomové práci.

_____ Datum převzetí zadání

Bc. Navara Lukáš

_____ Podpis studenta

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Navara** Jméno: **Lukáš** Osobní číslo: **502720**
Fakulta/ústav: **Fakulta elektrotechnická**
Zadávající katedra/ústav: **Katedra kybernetiky**
Studijní program: **Kybernetika a robotika**

II. ÚDAJE K DIPLOMOVÉ PRÁCI

Název diplomové práce:

Návrh recepčního robota řízeného velkým jazykovým modelem pro sociální interakci

Název diplomové práce anglicky:

Design of an LLM-Driven Receptionist Robot for Social Interaction

Pokyny pro vypracování:

The aim of this master's thesis is to investigate how large language models (LLMs) can support natural spoken interaction with a humanoid robot in a social setting.

Tasks

1. Review relevant literature on LLMs and social human–robot interaction.
2. Design and implement an LLM-based conversational framework to be deployed on the humanoid robot Pepper.
3. Integrate speech recognition and speech synthesis for spoken dialogue.
4. Implement retrieval-augmented question answering over selected internal documents.
5. On the robot Pepper, accompany the speech with appropriate non-verbal communication such as gestures and possibly facial expressions. Make appropriate use of the integrated tablet to display relevant information.
6. Demonstrate the system in a receptionist-style interaction scenario.
7. Compare cloud-based and locally deployed LLM backends in the implemented system, analysing their impact on latency and interaction quality.
8. Evaluate the interaction using established HRI questionnaires and user feedback

Seznam doporučené literatury:

- [1] Bartneck, C., Belpaeme, T., Eyssel, F., Kanda, T., Keijsers, M., & Šabanović, S. (2024). Human-robot interaction: An introduction. Cambridge University Press.
- [2] Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., Sermanet, P., Jackson, T., Brown, N., Luu, L., Levine, S., Hausman, K., & Ichter, B. (2023). Inner monologue: Embodied reasoning through planning with language models. Proceedings of the 6th Conference on Robot Learning (CoRL 2022), 205, 1769–1782. PMLR.
- [3] Ichter, B. A., Brohan, A., Chebotar, Y., Finn, C., Hausman, K., Herzog, A., ... Fu, C. K. (2023). Do As I Can, Not As I Say: Grounding language in robotic affordances. Proceedings of the 6th Conference on Robot Learning (CoRL 2022), 205, 287–318. PMLR.
- [4] Mower, C. E., Wan, Y., Yu, H., Grosnit, A., Gonzalez-Billandon, J., Zimmer, M., Wang, J., Zhang, X., Zhao, Y., Zhai, A., Liu, P., Tateo, D., Cadena, C., Hutter, M., Peters, J., Tian, G., Zhuang, Y., Shao, K., Quan, X., Hao, J., Wang, J., & Bou-Ammar, H. (2024). ROS-LLM: A ROS framework for embodied AI with task feedback and structured reasoning.
- [5] Qu, J., Jarosz, M., & Sniezynski, B. (2024). Robot control platform for multimodal interactions with humans based on ChatGPT. Applied Sciences, 14(17), 8011.

DECLARATION

I, the undersigned

Student's surname, given name(s): Navara Lukáš
Personal number: 502720
Programme name: Cybernetics and Robotics

hereby declare that the Master's Thesis titled

Design of an LLM-Driven Receptionist Robot for Social Interaction

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 16.05.2026

Bc. Lukáš Navara

.....
student's signature

Acknowledgements

I would like to thank doc. Mgr. Matěj Hoffmann, Ph.D., for his guidance and for maintaining such a positive atmosphere in the lab. I also thank MSc. Jason Khoury for his help with the experimental design, Ing. Lukáš Rustler for resolving connectivity issues, and Shubhan Parag Patni, MSc. for reviewing the thesis.

Abstract

This thesis designs an LLM-driven receptionist on the Pepper robot. A cascaded speech-to-text, large language model, and text-to-speech pipeline handles spoken dialogue; a set of helper functions looks up live faculty data such as staff contacts and lecture schedules; and body gestures and a live tablet transcript accompany the replies. The same pipeline supports two interchangeable backends, one fully local on a single GPU and one cloud-hosted.

The system was deployed for one day at a faculty reception desk, alternating the two backends across visitors. Twenty-six visitors completed a questionnaire of four Godspeed subscales, three custom Likert items, and preference questions. The cloud cascade received higher subjective ratings; the local cascade was faster to first audio; 19 of the 26 preferred the robot over a human at the desk. Because the two conditions differed in speech recognition, language model, and speech synthesis at once, the results read as an exploratory comparison of complete cascades rather than of language models alone. The work also weighs the two backends on upgradability, cost, and data handling.

Keywords: humanoid robot; large language model; human–robot interaction; voice agent; receptionist robot; local versus cloud deployment; Pepper

Abstrakt

Tato práce navrhuje recepčního robota řízeného velkým jazykovým modelem na platformě Pepper. Kaskáda převodu řeči na text, velkého jazykového modelu a syntézy řeči zpracovává hlasový dialog; sada pomocných funkcí vyhledává aktuální fakultní data jako kontakty na zaměstnance a rozvrh přednášek; odpovědi doprovázejí gesta a živý přepis konverzace na tabletu. Stejná pipeline podporuje dva zaměnitelné backendy, jeden plně lokální na jediném GPU a jeden hostovaný v cloudu.

Systém byl na jeden den nasazen na fakultní recepci, kde se mezi návštěvníky střídaly oba backendy. Dvacet šest návštěvníků vyplnilo dotazník složený ze čtyř subškál Godspeed dotazníku, tří vlastních Likertových položek a otázek na preferenci. Cloudová kaskáda obdržela vyšší subjektivní hodnocení; lokální kaskáda byla rychlejší k první zvukové odezvě; 19 z 26 respondentů dalo na recepci přednost robotovi před člověkem. Protože se obě varianty lišily v rozpoznávání řeči, jazykovém modelu i syntéze řeči zároveň, výsledky je třeba číst jako explorativní srovnání kompletních kaskád, nikoli jazykových modelů samotných. Práce dále zvažuje oba backendy z hlediska budoucí rozšiřitelnosti, ceny a nakládání s daty.

Klíčová slova: humanoidní robot; velký jazykový model; interakce člověka s robotem; hlasový agent; recepční robot; lokální vs. cloudové nasazení; Pepper

Překlad názvu: Návrh recepčního robota řízeného velkým jazykovým modelem pro sociální interakci

Contents

1 Introduction	1		
1.1 Motivation	1		
1.2 Goals	2		
1.3 Structure of the Thesis	2		
2 Related Work	5		
2.1 Receptionist and Guide Robots	5		
2.2 LLMs in Social Robotics	5		
2.3 Gestures in Human–Robot Interaction	7		
2.4 Evaluation of Social Robotics Interaction	8		
3 Background	9		
3.1 Large Language Models	9		
3.1.1 Tokenization and Representation	9		
3.1.2 Autoregressive Generation	10		
3.1.3 Context Window	10		
3.1.4 Practical Limitations	10		
3.2 Adapting LLMs: Prompting, Tools, and Retrieval	10		
3.2.1 Fine-Tuning	11		
3.2.2 Prompt Engineering	11		
3.2.3 Tool Calling	12		
3.2.4 Retrieval-Augmented Generation (RAG)	13		
3.3 Speech Interfaces for Human–Robot Interaction	13		
		3.3.1 Modular Cascaded Speech Architecture	13
		3.3.2 Integrated End-to-End Audio Architecture	14
		3.4 LLM Deployment: Local vs. Cloud	14
		3.4.1 Local Deployment	15
		3.4.2 Cloud-Based Deployment	15
		4 Materials and Methods	17
		4.1 System Overview	18
		4.2 Architecture Decisions	18
		4.2.1 Audio Playback	18
		4.2.2 Custom Acoustic Echo Cancellation	18
		4.2.3 Streaming Reply, Gestures via Tool Calls	19
		4.2.4 Tools and Prompt as the Only Extension Points	20
		4.3 Hardware Platform	20
		4.3.1 Pepper	20
		4.3.2 Compute Infrastructure	21
		4.4 System Architecture	21
		4.4.1 Services	21
		4.4.2 Communication	21
		4.5 Voice Interaction Pipeline	22
		4.5.1 Condition A: Local Cascade	23
		4.5.2 Condition B: Cloud Cascade	23
		4.5.3 Turn-Taking	23

4.6 Agent Behavior: Prompt and Tools	24	7.2 Implications	40
4.6.1 System Prompt	24	8 Future Work	43
4.6.2 Tool Surface	24	8.1 Implementation Fixes Surfaced by the Field Study	43
4.7 Non-Verbal Behaviour	26	8.2 Larger Directions	43
4.7.1 Gestures	26	Bibliography	45
4.7.2 Tablet	26	A Visitor Instruction Board	49
4.8 Session Management	27	B Post-Interaction Questionnaire	53
4.8.1 Experimental Loop	27		
4.9 Implementation and Deployment	27		
5 Experiments and Results	29		
5.1 Research Questions and Design .	29		
5.2 Procedure	30		
5.2.1 Setting	30		
5.2.2 Visitor Flow and Recruitment	30		
5.2.3 Ethics and Data Handling . . .	30		
5.3 Questionnaire	31		
5.4 Quantitative Results	32		
5.4.1 Sample	32		
5.4.2 Objective Latency	32		
5.4.3 Subjective Ratings	34		
5.4.4 Visitor Preferences	35		
5.5 Observations from Experiment .	35		
6 Conclusion	37		
7 Discussion	39		
7.1 Limitations	39		

Chapter 1

Introduction

1.1 Motivation

Robotic receptionists have been tested in public places for more than two decades, beginning with the CMU Roboceptionist Valerie [1] and continued in long deployments such as the Robovie field study in a Japanese shopping mall [2]. These earlier systems showed that people accept humanoid robots in real settings, but the dialogue itself was limited to scripted exchanges and rigid command menus. The language layer, not the embodiment, was the main bottleneck.



Figure 1.1: The CMU Roboceptionist Valerie, one of the earliest deployed receptionist robots [1].

Large language models (LLMs) change this picture by addressing the dialogue bottleneck that held the older systems back. For a receptionist this is a big shift: a visitor can simply ask a question instead of choosing from a menu, and the robot can ground its reply in

voice backends compared in this work (a locally hosted cascade and a cloud-hosted cascade), the knowledge-grounding layer that lets the robot answer questions about the institution, and the gesture and tablet feedback shown during the conversation. Chapter 5 presents the user study: the between-subjects design contrasting the two backends, the recruitment and procedure at the chosen reception desk, the questionnaire and custom items used to measure the interaction, and the objective, subjective, and qualitative results. Chapter 6 gives a short summary of what was built and what was found. Chapter 7 discusses the limitations of the work and the implications of the results. Chapter 8 outlines directions for further work.



Figure 1.2: The deployed system: a visitor talking to Pepper at the reception desk.

Chapter 2

Related Work

2.1 Receptionist and Guide Robots

As noted in the motivation (Section 1.1), two pre-LLM deployments anchor this work: Valerie the Roboceptionist at Carnegie Mellon University (2005) [1] and Robovie in a Japanese shopping mall (2010) [2]. Both predate LLM-driven dialogue and make concrete what was missing before.

Valerie was placed in a booth in a busy hallway of the Computer Science building at CMU, where she greeted visitors, gave directions, and told scripted “stories” about her own fictional life. The system deliberately used text input from visitors at the booth rather than speech recognition, because off-the-shelf speech recognition of the time was not robust in a noisy hallway [1]. The dialogue was authored in advance, and the team explicitly identifies the resulting brittleness as a long-term design problem: Valerie would continue telling her stories after visitors had departed, the interaction style did not adapt for returning visitors, and any topic outside the scripted set had no graceful answer.

Robovie was deployed for a 25-day field trial in a Japanese shopping mall, where its tasks were shopping information, route guidance, and building rapport with visitors [2]. The architecture was a “network robot”: most behaviors were pre-scripted, and a human operator could intervene by typing free-form text when the robot encountered an unexpected question. A survey reported in the paper found that 64.2% of respondents considered robots appropriate for commercial places and 87.9% considered guidance an appropriate robotic task, which suggests that public acceptance of robotic information desks is not the blocking factor.

Both deployments show that placing a humanoid at a public reception or information desk is workable and that visitors are willing to engage. They also show the main failure mode of the pre-LLM era: any utterance outside the authored script either fell back to a pre-programmed reply or required a human operator.

2.2 LLMs in Social Robotics

The use of a large language model as the dialogue backend of a social robot is recent, but enough early studies exist to mark out the shape of the field. The two studies closest to the present thesis both attach a LLM to a Pepper robot and place the result in front of users. Chen *et al.* (2024) [3] replaced Pepper’s limited built-in keyword speech recognition with



Figure 2.1: Robovie deployed in a Japanese shopping mall, giving route guidance to visitors [2].

Whisper [4] for transcription and routed the user’s words through ChatGPT for the reply, with the explicit aim of testing whether a more capable language layer makes the robot feel more relatable. They report an interesting limitation that is worth carrying into any later study of the same form: participants with extensive prior ChatGPT experience held higher expectations of the robot and were more easily disappointed when those were not met. Herath *et al.* (2025) [5] deployed a comparable Pepper–ChatGPT system at a public science festival and gathered first impressions from the visitors. A recurring complaint in their data was the response latency (“the response time of 4 s made it hard to interact”), which shows that latency is a real engineering problem and not only a benchmark number.

A more directly comparable system is the one by Qu *et al.* (2024) [6], who built a ChatGPT-based control platform for multimodal interaction with humans on a humanoid robot. They wired the LLM into the dialogue loop alongside speech recognition and synthesis, and reported that the resulting interaction felt more natural than a scripted baseline.

The closest sibling system in the literature is FurChat by Cherakara *et al.* (2023) [7], a receptionist robot at the National Robotarium. They deployed GPT-3.5 on a Furhat robot and combined an open-domain dialogue mode with a closed-domain mode backed by a small dictionary scraped from the host institution’s website. The LLM also picked an emoticon for each reply, which was mapped to one of Furhat’s pre-defined facial gestures. FurChat was presented as a demonstrator rather than evaluated with users, but it showed that a single LLM could handle both factual questions about a venue and casual smalltalk, with the embodiment layer (facial expressions) generated by the same reasoning step.

The idea of an LLM controlling a robot through a small set of tools (see Section 3.2.3) predates these social robotics applications. Huang *et al.* (2022) [8] introduced an “inner

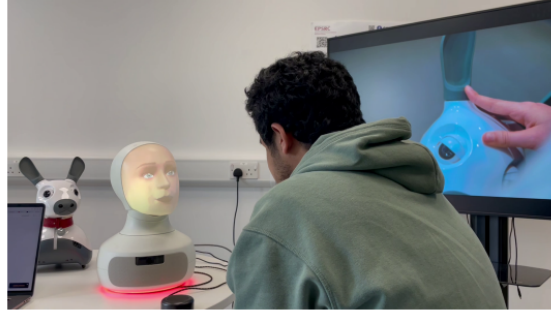


Figure 2.2: FurChat on the Furhat robot, with an LLM-selected facial expression accompanying the reply [7].

monologue” loop in which the language model alternates between planning the next step and reading feedback from the robot’s perception and skills, and Ahn *et al.* (2022) [9] filtered LLM action proposals through learned estimates of which actions the robot could actually carry out on a mobile manipulator. Neither targeted a humanoid receptionist, but they established the LLM-plus-validated-tool interface that this thesis applies to a social interaction role.

2.3 Gestures in Human–Robot Interaction

How a robot uses its body during interaction—gestures, gaze, posture—shapes how a person reads its behavior. Saunderson *et al.* (2019) [10] survey this literature and document robust effects on engagement, likeability, perceived intelligence, and trust across many platforms and tasks.

A more targeted question—whether the gestures need to match the speech exactly to be useful—is addressed by Salem *et al.* (2013) [11] in a controlled study. They compared three conditions on a humanoid robot: speech only; speech with gestures that matched the content; and speech with gestures that were occasionally incongruent with the content. Both gesture conditions raised perceived anthropomorphism over speech alone, and—the surprising finding—the incongruent-gesture condition produced significantly higher likability than the speech-only condition. In other words, imperfect gestures were still better than no gestures.

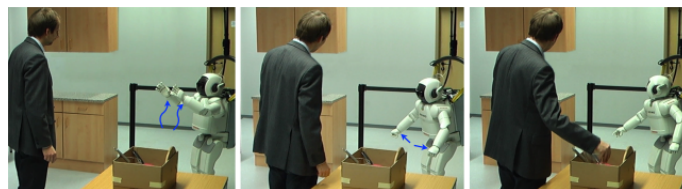


Figure 2.3: The humanoid robot used in the Salem *et al.* study comparing speech-only, congruent-gesture, and incongruent-gesture conditions [11].

2.4 Evaluation of Social Robotics Interaction

Interaction quality in a social robotics setting is hard to read from logs alone. Two visitors can leave a session with the same task completed and the same conversation length, yet experience the robot very differently—one engaged, one frustrated. This gap is the reason questionnaire-based instruments are the default way to evaluate social robots, even when the system also produces detailed objective measures from its logs.

The most widely used instrument is the Godspeed questionnaire of Bartneck *et al.* (2009) [12], which measures five separate attributes on bipolar adjective scales: anthropomorphism (humanlike versus machinelike), animacy (how lifelike), likeability (how friendly), perceived intelligence, and perceived safety. Each subscale yields a separate mean score and the instrument has been used widely enough that comparison across studies is straightforward. Carpinella *et al.* (2017) [13] later proposed the Robotic Social Attributes Scale (RoSAS) as a complement to Godspeed, reorganizing the social attribution literature into three factors: warmth, competence, and discomfort. The RoSAS authors explicitly position their scale as additive rather than as a replacement, so the two are often reported together.

A different angle on evaluation is taken by the Almere model of Heerink *et al.* (2010) [14], which extends the Technology Acceptance Model to assistive social robots and asks not what the robot is perceived to be, but whether the user intends to use it again. Acceptance and perception capture different things, and which one matters depends on the deployment goal: a one-off public installation is well served by perception scales, while a long-term assistive deployment is better evaluated through an acceptance lens. The choice of instrument made for the present thesis is described in Chapter 5.

Chapter 3

Background

This chapter gives a short overview of the concepts that the rest of the thesis builds on: how large language models work and how their behavior is shaped, how speech is handled in human–robot dialogue, and the trade-offs between locally hosted and cloud-hosted model deployment. The aim is not a complete treatment but enough conceptual grounding for the design choices in Chapter 4.

3.1 Large Language Models

Large Language Models (LLMs) are neural networks trained on large text corpora to predict the next token in a sequence. Formally, an LLM estimates the conditional probability distribution

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}),$$

the probability of the next token given all previously observed tokens.

Modern LLMs are based on the Transformer architecture [15], which replaces recurrent computation with self-attention.

3.1.1 Tokenization and Representation

Before text can be processed by a model, it must be converted into discrete units called *tokens*. Tokens are not necessarily whole words; they may correspond to subword fragments, characters, or punctuation.

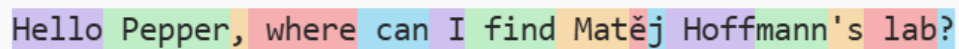


Figure 3.1: Example tokenization produced by the OpenAI Tokenizer tool [16].

Each token is mapped to a high-dimensional continuous vector representation called an *embedding*. These embeddings capture semantic similarity: tokens appearing in similar contexts obtain nearby representations in vector space.

■ 3.1.2 Autoregressive Generation

LLMs operate autoregressively. Given an input prompt (e.g., a user query), the model generates output one token at a time. At each step, it computes a probability distribution over the vocabulary and samples or selects the next token. This token is appended to the sequence and becomes part of the context for the next prediction.

Complex behaviors such as summarization, dialogue, or planning emerge from this objective at scale [17].

■ 3.1.3 Context Window

The *context window* defines the maximum number of tokens the model can process at once. All dialogue history, system instructions, retrieved documents, and intermediate tool outputs must fit within this window. If the limit is exceeded, older tokens must be truncated or summarized.

Context limitations directly affect system design in long-running interactions and motivate external memory mechanisms such as retrieval augmentation [18].

■ 3.1.4 Practical Limitations

Despite their capabilities, LLMs exhibit several well-known limitations:

- **Hallucinations:** The model may generate fluent but factually incorrect information when the prompt lacks grounding [19].
- **Non-determinism:** Due to probabilistic sampling and internal stochasticity, outputs may vary across runs. The variability can be reduced through the temperature parameter but not eliminated. For some uses, such as creative writing, the variability is welcome.
- **Limited world state awareness:** The model does not have real-time knowledge unless external information is explicitly provided.
- **Latency and computational cost:** Token-by-token generation introduces delay, which becomes significant in real-time robotic interaction.

■ 3.2 Adapting LLMs: Prompting, Tools, and Retrieval

Out of the box, an LLM produces free-form text and does not automatically stay within a task definition or ground its answers in external information. Four complementary strategies address this: fine-tuning, prompt engineering, tool calling, and retrieval-augmented generation.

■ 3.2.1 Fine-Tuning

Fine-tuning adapts a pre-trained language model by updating its weights on additional task-specific data. Instead of training from scratch, a base model is further optimized on curated instruction datasets, dialogue transcripts, or domain-specific data. This approach enables specialization of tone, behavior, and knowledge distribution.

Instruction tuning and reinforcement learning from human feedback (RLHF) have been widely used to align models with desired conversational norms and safety constraints [20,21]. Fine-tuning can significantly improve task performance and reduce undesirable behaviors in well-defined domains.

However, fine-tuning introduces important tradeoffs:

- **Data demand:** A substantial amount of high-quality, task-relevant data is required.
- **Cost:** Training large models requires substantial compute resources.
- **Maintenance:** Updates to institutional knowledge require repeated retraining.
- **Reproducibility:** Stochastic training processes and proprietary base models complicate exact replication.

In domains where institutional knowledge changes frequently (e.g., room assignments, contact information, schedules), embedding such volatile data directly into model weights is inefficient and hard to maintain. Systems in such domains typically rely on prompting and external tools for behavioral control and knowledge access rather than on fine-tuning.

■ 3.2.2 Prompt Engineering

Prompt engineering refers to the structured design of input instructions that guide model behavior without modifying its internal parameters. Modern LLMs respond strongly to in-context examples and to reasoning demonstrations provided in the prompt [22,23].

System-level instructions can define persona, tone, safety rules, and output structure. Few-shot prompting provides task demonstrations that bias the model toward desired patterns of reasoning or formatting. Output constraints (e.g., JSON schemas) further restrict generation space and improve downstream reliability.

Prompt-based control has several advantages:

- No additional training cost.
- Immediate iteration and deployment.
- Separation between base model and application logic.

However, prompting does not inject new factual knowledge into the model. It can steer style and reasoning patterns, but it cannot reliably compensate for missing or outdated information. This limitation motivates the integration of external knowledge sources.

3.2.3 Tool Calling

Tool calling (also referred to as function calling) enables an LLM to produce structured outputs that trigger executable actions instead of free-form text [24–26]. Rather than generating only natural language, the model may return a structured function call with validated arguments that correspond to a predefined interface, as formalized in modern LLM APIs [24].

This mechanism separates reasoning from execution. The LLM decides *what* should be done, while external deterministic code performs *how* it is done. This is critical in embodied systems, where actions must be reliable, safe, and constrained.

Figure 3.2 illustrates the typical interaction loop, and the process consists of the following steps:

1. The developer defines available tools (e.g., `get_weather(location)`) and provides them to the model together with the user message.
2. The model decides that answering requires structured data and produces a tool call instead of a textual reply.
3. The application executes the corresponding function in deterministic code and obtains a result.
4. The result is appended to the conversation context.
5. The model generates the final user-facing response grounded in the returned data.

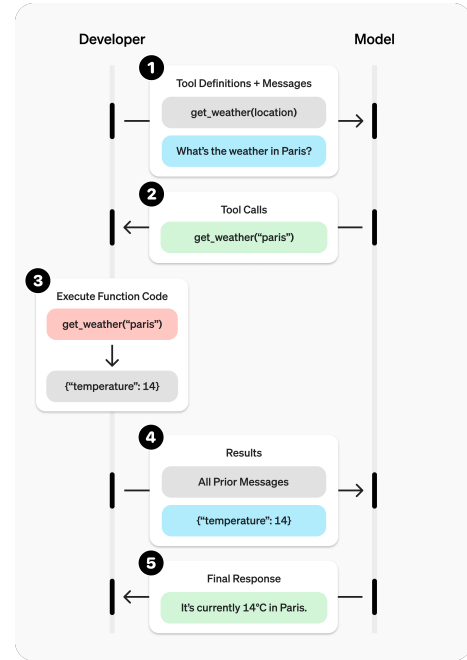


Figure 3.2: Typical interaction loop of an LLM tool call. Source: OpenAI Function Calling Guide [24].

This structured interaction reduces hallucinations because factual information is retrieved from verified sources rather than generated from internal model memory.

Tool implementations in embodied systems typically fall into two categories: knowledge-retrieval tools that query external sources (documents, contacts, schedules), and robot-control tools that trigger physical actions (navigation, gestures, gaze). Both share the same

structured calling mechanism, which simplifies orchestration and ensures that all external effects are mediated through validated interfaces.

■ 3.2.4 Retrieval-Augmented Generation (RAG)

A baseline approach to grounding an LLM in domain-specific knowledge is to insert all relevant documents directly into the context window. However, this strategy does not scale due to:

- Context window limits,
- Increased token cost,
- Noisy or irrelevant information degrading output quality.

Retrieval-Augmented Generation (RAG) addresses this limitation by separating storage from reasoning [18]. The typical pipeline consists of:

1. Encoding documents into vector embeddings,
2. Indexing them in a vector database,
3. Retrieving semantically relevant chunks at query time,
4. Conditioning the LLM response on the retrieved context.

Instead of permanently embedding knowledge into model weights, RAG dynamically injects only the most relevant information into the prompt. This improves factual grounding and reduces hallucination risk.

A common implementation pattern in embodied systems exposes retrieval as a tool call triggered during dialogue, sharing the same structured interface as robot-control tools. This creates a unified orchestration layer in which language reasoning, information grounding, and physical action share a common mechanism.

■ 3.3 Speech Interfaces for Human–Robot Interaction

A spoken-dialogue system must process audio input, perform language reasoning, and generate speech output in real time. Two architectures are used in practice: the modular cascaded pipeline and the integrated end-to-end audio model.

■ 3.3.1 Modular Cascaded Speech Architecture

The modular cascaded speech architecture decomposes spoken interaction into three sequential stages: Speech-to-Text (STT), language reasoning using a Large Language Model (LLM) and Text-to-Speech (TTS) synthesis.

In this pipeline, user speech is first transcribed into text by the STT module. The textual transcript is then processed by the LLM, which performs dialogue reasoning and may invoke tool calls. Finally, the generated textual response is synthesized into audio by the TTS system and played back to the user.

This architectural separation provides several advantages:

- **Modularity:** Each component can be independently developed, replaced, or optimized.
- **Transparency:** The intermediate textual representation enables logging, inspection, and debugging.

However, the cascaded design also introduces limitations:

- **Cumulative latency:** Delays from STT, LLM generation, and TTS synthesis accumulate sequentially.
- **Error propagation:** Transcription errors in STT directly affect downstream reasoning.
- **Pipeline orchestration complexity:** Real-time coordination between independent modules requires additional engineering effort.

■ 3.3.2 Integrated End-to-End Audio Architecture

Integrated end-to-end audio architectures model speech perception, language reasoning, and speech generation within a unified multimodal framework. Recent multimodal foundation models operate directly on audio representations, enabling speech-in to speech-out interaction without mandatory intermediate textual stages.

Key characteristics include:

- unified perception and generation
- reduced explicit pipeline orchestration
- potentially lower end-to-end latency
- native multimodal reasoning capabilities

The absence of an intermediate textual representation reduces transparency and complicates debugging.

■ 3.4 LLM Deployment: Local vs. Cloud

Large language models and multimodal foundation models can be deployed either locally on dedicated hardware or accessed through cloud-based APIs. The two options differ in cost structure, performance, scalability, and data governance.

■ 3.4.1 Local Deployment

Local deployment involves running models on privately controlled hardware, typically GPU-equipped servers. Inference performance depends directly on available computational resources, including GPU memory capacity, compute throughput, and storage bandwidth. Larger models generally provide improved reasoning quality and robustness, but require substantially more memory and computational power.

The primary characteristics of local deployment include:

- **Hardware dependency:** Model size is constrained by available GPU memory and compute capability.
- **Performance–quality trade-off:** Smaller models enable lower latency and reduced resource consumption, while larger models often provide higher response quality at increased computational cost.
- **Fixed infrastructure cost:** After hardware acquisition, inference does not incur per-token or per-request fees.
- **Data sovereignty:** All data processing occurs within controlled infrastructure, reducing exposure of sensitive information.

Local deployment is particularly attractive in scenarios involving sensitive data, strict privacy requirements, or predictable high interaction volume. However, it requires maintenance of hardware infrastructure, model updates, and system optimization.

■ 3.4.2 Cloud-Based Deployment

Cloud-based deployment provides access to large-scale foundation models through remote APIs. The provider manages infrastructure, scaling, optimization, and model updates.

Key properties include:

- **Elastic scalability:** Compute resources scale automatically based on demand.
- **Access to frontier-scale models:** Very large models may be accessible without requiring local hardware.
- **Usage-based pricing:** Costs are typically proportional to the number of processed tokens or requests.
- **Reduced operational overhead:** Infrastructure management and optimization are handled by the provider.

Recurring token-based pricing can become significant under high interaction volume. Data must also be transmitted to external servers, which may raise privacy or regulatory concerns depending on the application domain.

Chapter 4

Materials and Methods

All code used in this thesis is available at the project repository <https://github.com/navarlu/Pepper> [27]; file and directory paths in this chapter are given relative to the repository root. This chapter is organized so that the overview comes first, the non-obvious design choices come next, and the lower-level details follow. Section 4.1 gives a high-level picture of what runs where. Section 4.2 explains the key architecture decisions that shaped the system. The remaining sections (Section 4.3 – Section 4.9) fill in the hardware, the service architecture, the voice pipeline, the agent’s behavior, the non-verbal channel, the session lifecycle, and the deployment details.



Figure 4.1: The deployed system at the reception. Pepper stands behind the desk and the Raspberry Pi sits next to her on the staff side.

4.1 System Overview

The deployed system spans three physical machines (Figure 4.2). A Raspberry Pi 5 is located next to Pepper at the reception desk and hosts most of the supporting services: the LiveKit media server (see Section 4.4.2), the robot **bridge** that talks to NAOqi (see Section 4.3.1), an **audio-bridge** that pipes synthesized speech onto Pepper’s speakers, a **user-client** that captures the visitor’s microphone, a **tablet-server** that renders the on-chest tablet, the **experiment-orchestrator**, a per-session driver, and a few utility services (startup watchdog, reverse SSH tunnel, vector database). A departmental GPU server hosts the language-model inference (vLLM serving Llama 3.1 8B Instruct) and runs the two **voice-agent** processes themselves. Pepper itself embodies the agent: its speakers carry the synthesized voice, its chest tablet displays dialogue state and tool results, and its animated body provides the gestures the visitor sees.

The split is driven by hardware: the local voice cascade needs a GPU that the Raspberry Pi does not have, and the GPU server is on the faculty network which cannot reach the Raspberry Pi inbound, so an **autossh**-managed reverse SSH tunnel is opened from the Raspberry Pi to the GPU server and the LiveKit ports are forwarded back over it. The cloud cascade is hosted on the same GPU server for operational uniformity, even though it does not need the GPU itself.

Three transports glue everything together: WebRTC via LiveKit for the real-time audio between the visitor, the voice agent, and the playback **bridge**; HTTP REST for tool calls, gesture dispatch, and tablet updates; and a long-lived SSH connection feeding **paplay** on Pepper for the actual playback (see Section 4.2.1).

4.2 Architecture Decisions

This section explains the four design choices that most shaped the deployed system. Each of them was made under a concrete constraint—latency, the location of microphone and speaker, or the assignment’s requirements—and each has visible consequences in the rest of the chapter.

4.2.1 Audio Playback

An earlier version of the audio bridge fed PCM frames over a TCP socket into NAOqi’s **ALAudioDevice** service. That path added roughly one second of playback buffer that could not be reduced from outside NAOqi, which dominated the perceived response time. The deployed system therefore bypasses NAOqi for real-time TTS playback: the **audio-bridge** service spawns a persistent **ssh + paplay** subprocess into Pepper’s PulseAudio daemon and writes the agent’s synthesized audio directly to its stdin (`services/src/live/audio_bridge.py`).

4.2.2 Custom Acoustic Echo Cancellation

The microphone is on the visitor’s side of the reception desk; the speakers are on Pepper, on the other side of a pane of glass. Without echo cancellation, the microphone picks

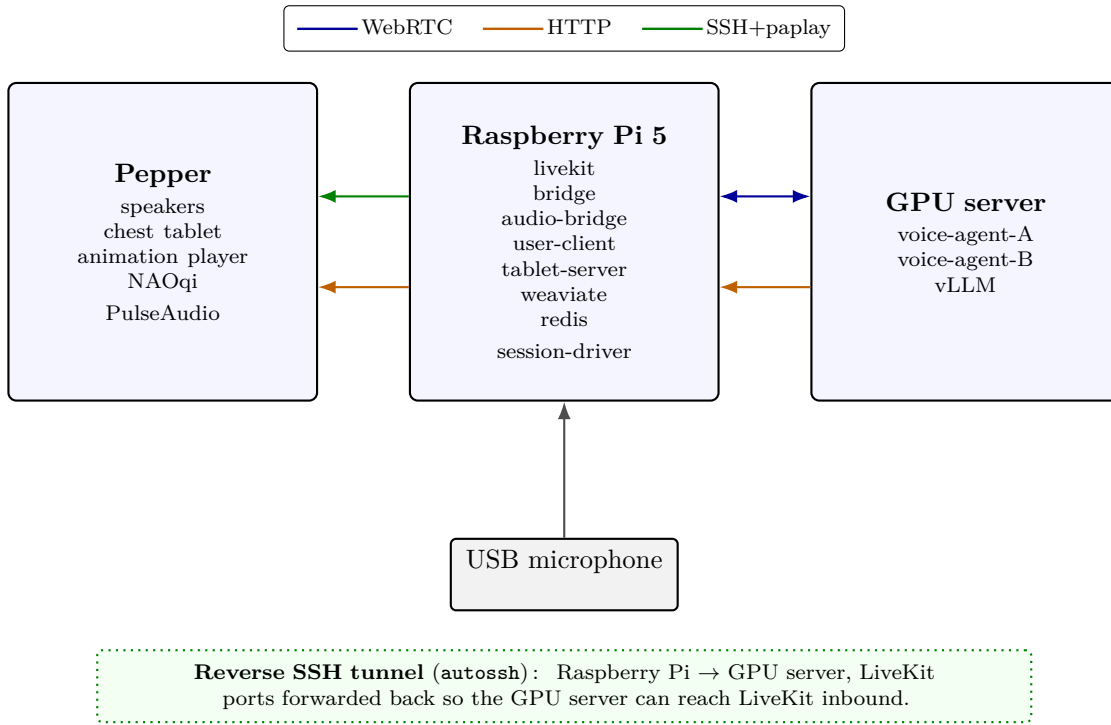


Figure 4.2: System architecture. Three physical machines and the transports between them. WebRTC carries real-time audio (LiveKit room); HTTP REST carries tool calls from the voice agent to the **bridge** and animation/tablet updates from the **bridge** to NAOqi; SSH+**paplay** streams the agent’s synthesized reply directly to Pepper’s PulseAudio daemon, bypassing NAOqi (see Section 4.2.1). The Raspberry Pi initiates an **autossh**-managed reverse tunnel to the GPU server because the GPU server cannot reach the Raspberry Pi inbound.

up Pepper’s own voice loud enough that the voice activity detector treats it as a visitor turn, so the agent ends up replying to itself. An early workaround was to mute the visitor’s microphone while Pepper was speaking; this prevented the self-reply but made the interaction feel unnatural, because visitors could not interrupt or speak over her. LiveKit’s built-in echo cancellation would have been the obvious alternative, but it assumes that microphone and speaker share one physical device, which is not the case here. Echo cancellation is therefore done in the **user-client** service before the microphone audio is published into the LiveKit room: a WebRTC AEC3 module is fed both the microphone capture and the agent’s reply track as a reference, and the cleaned-up signal is what the speech-to-text component sees. With the echo removed, the visitor can interrupt Pepper mid-reply without her own voice being mistaken for the interruption.

4.2.3 Streaming Reply, Gestures via Tool Calls

An earlier design followed the tool-only pattern of the Letta agent framework [28]: the language model could only speak by calling a `send_message_to_user(text="", emotion="")` tool with the reply text and a gesture label as arguments. This had the appealing property that every spoken sentence carried an explicit gesture chosen by the same reasoning step that produced the text, so the visible body language matched the words in a fine-grained

way. The problem was latency: a tool call cannot be issued until the language model has finished writing its arguments, so the entire reply text has to be generated before the text-to-speech component sees the first character.

The deployed system streams the reply directly: the language model emits tokens, the text-to-speech component consumes them as they arrive, and Pepper begins to speak as soon as the first sentence is ready. Gestures still exist, but they are dispatched only when the model calls one of the tools (`lookup`, `navigation`, `schedule`, `time`, `query_search`, `end-of-conversation`), each of which takes in addition to its task-specific arguments (e.g., `lookup(surname, emotion)`). The trade-off is honest: the surface of gesture intents is narrower than under the tool-only design—most gestures are now “thinking” on the way to a tool result, and the open-ended “explaining” or “offering” gestures that the older design could produce during the reply itself are missing. First-audio latency was the deciding factor.

■ 4.2.4 Tools and Prompt as the Only Extension Points

Pepper’s behavior and knowledge are defined entirely by two things: the system prompt and the set of registered tools. Changing the persona means editing the identity and personality sections of the prompt; adding a new capability—a database lookup, a new robot motion, an integration with a third-party service—means writing one Python function, decorating it with the LiveKit Agents `@function_tool` decorator, and naming it in the prompt. Nothing else in the pipeline needs to change.

The same modularity extends to the dialogue model itself. In Condition A the language model runs on the GPU server under vLLM, and switching to a different local model is a matter of pointing vLLM at another checkpoint. In Condition B the cloud model is selected by a single configuration string, so swapping `gpt-4o-mini` for any other OpenAI chat-completion model is a one-line change.

■ 4.3 Hardware Platform

■ 4.3.1 Pepper

Pepper is a 1.2 m tall humanoid robot [29]. All on-board sensors, actuators, speakers, and the chest tablet are accessed through NAOqi, middleware that ships with the robot [30]. The receptionist scenario uses Pepper’s speakers, the chest-mounted tablet for the current-state display, and the animation player for gesture playback. Pepper’s autonomous-life service is left enabled while the robot is idle so that small head and body movements give a lifelike presence; during a session the head is locked to a forward-and-slightly-upward pose so that the apparent gaze stays on the visitor.

The robot’s onboard microphone array is not used;

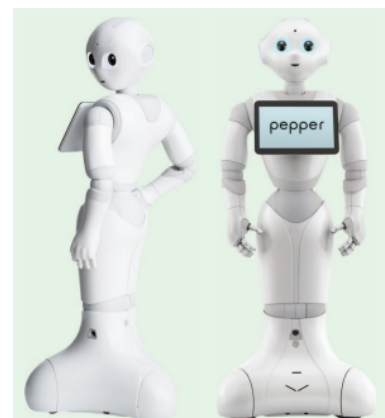


Figure 4.3: The Pepper humanoid robot [29].

speech is captured with an external USB microphone connected to the Raspberry Pi. One bottom laser-range sensor on the particular Pepper unit developed a fault during the project and caused the start-up self-test to fail; a small `safe-startup` service suppresses the offending check and clears the resulting error notification on every boot.

4.3.2 Compute Infrastructure

Two compute nodes run the system in addition to Pepper itself:

1. **Raspberry Pi 5 8GB RAM.** Sits next to Pepper at the reception desk. It runs a local wireless hotspot that Pepper joins on startup, and is also connected to the faculty network. The Raspberry Pi hosts the LiveKit media server, the supporting services listed in Table 4.1, and the per-session driver.
2. **Departmental GPU server.** Reachable through the faculty network, equipped with an NVIDIA RTX 4000 SFF Ada Generation GPU [31]. It hosts both `voice-agent` processes—the local cascade described in Section 4.5.1 and the cloud cascade described in Section 4.5.2—together with a vLLM [32] server running Llama 3.1 8B Instruct [33] that the local cascade calls.

Because the GPU server cannot reach the Raspberry Pi inbound, a reverse SSH tunnel is opened from the Raspberry Pi. The LiveKit ports are forwarded over this tunnel, and the audio path of Section 4.2.1 reuses the same SSH connection.

4.4 System Architecture

The system is a set of loosely coupled services that talk to each other over three transports. The decomposition was driven by two practical needs: independent restartability during development and demonstrations, and the ability to run the voice agents on a different physical machine from the rest of the stack.

4.4.1 Services

Table 4.1 lists the services that make up the deployed system. Services on the Raspberry Pi run as containers managed by Docker Compose; the two voice agents and the vLLM language-model server on the GPU server, together with the session driver on the Raspberry Pi, run as plain Python processes inside `tmux` sessions so that they survive disconnections from the experimenter’s terminal.

4.4.2 Communication

Three transports bind the services together, each chosen for what it does well:

Service	Purpose
livekit	WebRTC media server and agent dispatch
redis	State backend for LiveKit
weaviate	Vector database used by <code>query_search</code> (Section 4.6.2)
experiment-orchestrator	Creates the LiveKit room and issues access tokens
bridge	Robot-control gateway (animation, tablet, LEDs, volume)
audio-bridge	Pipes the agent’s audio track into <code>ssh + paplay</code> on Pepper
user-client	Captures the USB microphone, runs AEC, publishes to LiveKit
tablet-server	Renders dialogue-state HTML and pushes it to Pepper’s tablet
safe-startup	One-shot pose and volume normalizer on Pepper boot
reverse-tunnel	<code>autossh</code> tunnel from RPi to the GPU server
vllm	Language-model inference server for Condition A (GPU server)
voice-agent-A	Local cascade voice agent (Section 4.5.1, GPU server)
voice-agent-B	Cloud cascade voice agent (Section 4.5.2, GPU server)
loop_launcher_streaming.py	Per-session driver; alternates Condition A and Condition B

Table 4.1: Services that make up the deployed system. All services run on the Raspberry Pi unless marked (GPU server).

1. **WebRTC via LiveKit.** LiveKit is an open-source WebRTC framework whose Agents SDK orchestrates the speech-to-text, language-model, and text-to-speech components and routes audio between the visitor and the agent [34]. Real-time audio between the visitor’s microphone, the voice agent, and the `audio-bridge` passes through the LiveKit media server. LiveKit was chosen for its room-level participant management and codec handling; echo cancellation is *not* delegated to LiveKit and is performed in the `user-client` instead (see Section 4.2.2).
2. **HTTP REST.** Tool calls from the language model, gesture dispatch, tablet updates, and miscellaneous control commands all use synchronous HTTP to the `bridge`.
3. **SSH + `paplay`.** The audio path from the agent’s synthesized reply to Pepper’s speakers, described in Section 4.2.1.

4.5 Voice Interaction Pipeline

The system supports two voice interaction backends, the two compared in this work. Both expose the same tool interface, session lifecycle, and robot-control interface to the rest of the architecture; they differ only in *where* the speech is processed and *which* models handle the dialogue.

Both backends are cascaded streaming pipelines: a voice activity detector segments the user’s turn, a speech-to-text component transcribes the utterance, a language model generates the reply token by token, and a text-to-speech component synthesizes the audio incrementally so that Pepper begins to speak as soon as the first sentence is ready. End-to-end speech-to-speech models such as Moshi [35] or the OpenAI Realtime API are an alternative architecture. Using cascades for both conditions is a study-design decision: it isolates the local-versus-cloud comparison from any confound introduced by an architectural difference between the two, and is not a claim that cascades are generally preferable to end-to-end audio models.

Both **voice-agent** worker processes are built on the LiveKit Agents SDK and share most of their wiring through a common pipeline-construction routine: the same agent class, the same tool surface, the same recording schema. They differ only in the concrete component instances chosen when the pipeline is built, and they register with LiveKit under different agent names so that the session driver can dispatch the correct one for each visitor.

■ 4.5.1 Condition A: Local Cascade

The local cascade runs on the GPU server. Its components are:

- **Voice activity detection.** A voice activity detector (VAD) flags whether each short audio frame contains speech, which lets the pipeline know when the visitor's turn starts and ends. We use Silero VAD [36], a small open-source neural detector.
- **Speech-to-text.** `faster-whisper` [4], a fast port of OpenAI's Whisper, running the small model.
- **Language model.** Llama 3.1 8B Instruct [33], served by vLLM [32]. Llama 3.1 8B was chosen because it is the largest model that fits on the available GPU together with the speech components and because its tool-calling ability is competitive with small frontier models on the Berkeley Function Calling Leaderboard [37].
- **Text-to-speech.** Piper, a fast open neural TTS engine, with an English female voice.

■ 4.5.2 Condition B: Cloud Cascade

The cloud cascade runs on the same GPU server as Condition A; it does not use the GPU itself. So it could run on the Raspberry Pi, but hosting it on the GPU server keeps the deployment more uniform and avoids adding extra load to the Raspberry Pi. Its components are:

- **Voice activity detection.** The same Silero VAD configuration as Condition A.
- **Speech-to-text.** OpenAI `gpt-4o-mini-transcribe`.
- **Language model.** OpenAI `gpt-4o-mini`, reached over the standard OpenAI REST API.
- **Text-to-speech.** OpenAI `gpt-4o-mini-tts`, using the `nova` voice with a short instruction asking the model to speak in a warm, conversational receptionist tone.

■ 4.5.3 Turn-Taking

Both backends use the voice activity detector to determine when the visitor has finished speaking. Once end-of-turn is declared, the cascade generates a reply. Visitors are free to interrupt Pepper mid-reply: with the echo cancellation of Section 4.2.2 in place, Pepper's own voice is removed from the captured microphone signal, so the voice activity detector

can still pick out a genuine visitor utterance while Pepper is speaking and cut her off cleanly.

■ 4.6 Agent Behavior: Prompt and Tools

The agent’s behavior is shaped by two surfaces: a *system prompt* that tells the language model who Pepper is and how to behave, and a *tool surface* that lets the language model act on the world. These two surfaces are the only places where domain knowledge and persona live; the rest of the pipeline is plumbing that does not change when the agent is adapted to a different scenario (see Section 4.2.4).

■ 4.6.1 System Prompt

The system prompt is a single text document organized into three named sections:

- **Identity, personality, and known facts**—who Pepper is, the tone she should adopt (warm, brief, slightly playful), and a short list of facility facts that are stable enough and short enough to include directly in the prompt (example room codes, opening hours of nearby facilities) so that she does not waste a tool call on them.
- **Reply style**—one or two short sentences, plain conversational prose, no markdown, no stage directions, and an explicit rule that the model never mentions tools or function names in its reply.
- **When to call tools**—a positive-framed trigger for each tool of the form “call *tool-name* when the user asks for X”.

The opening greeting is not part of the system prompt. The moment a session starts, Pepper speaks a fixed greeting line directly through the text-to-speech component, before the language model is called at all. During the experiment this turned out to be a mistake. The greeting plays unconditionally before the language model is involved, so a visitor who walked up and asked a question on the first turn got “Hello, how can I help you today?” instead of an answer.

■ 4.6.2 Tool Surface

The deployed agent exposes seven tools, listed in Table 4.2. Both Condition A and Condition B see exactly the same tool surface so that the resulting transcripts are directly comparable.

The initial design used `query_search` as the only knowledge backend: a Weaviate vector store would hold institutional documents and the agent would retrieve from them for any factual question. This design was inspired by *FELChat* [38], an earlier team project that answers questions from the same set of CTU and FEE documents, and the same corpus was uploaded into the Pepper deployment.

Tool	Purpose	Data source
<code>find_path_to_room</code>	Walking directions from the entrance to a given room code.	Manually prepared dictionary.
<code>lookup_person</code>	Office, phone, and email of a staff member by surname.	Live HTTPS scrape of the public faculty directory.
<code>subject_schedule</code>	Next session of a course, optionally filtered by type or day.	Live HTTP fetch of the public faculty timetable.
<code>mensa_menu</code>	Canteen menu for the current and next week.	Live HTTP fetch of the canteen page.
<code>get_time</code>	Current local time.	Local system clock.
<code>query_search</code>	Retrieval-augmented search over institutional documents.	Weaviate vector database.
<code>end_conversation</code>	Speaks a short farewell, resets the session.	Control-flow tool.

Table 4.2: The seven tools exposed to the language model. Both conditions share the same surface.

During development it became clear that the institutional information visitors actually need at a reception desk—room locations, staff contacts, course schedules, the canteen menu—is already available behind public faculty endpoints, and that fetching it directly is more reliable than retrieving paragraphs about it from a PDF. The system was therefore extended with three targeted live HTTP endpoints—`lookup_person`, `subject_schedule`, and `mensa_menu`—that fetch the current data on demand and return short, prompt-friendly results. The advantage over a static vector store is freshness: when a room is reassigned or a lecture moves to a different time, the next visitor automatically gets the new value with no maintenance step.

`query_search` remains in the tool surface for what the live endpoints do not cover: domain-specific knowledge such as internal regulations, institutional procedures, or faculty-specific documents. Extending Pepper along this axis is a single drop-in operation (details in the next paragraph).

The knowledge base used by `query_search` is a small set of CTU and FEE official documents: the Czech Higher Education Act 111/1998 (English version), the CTU Statute, and the Accommodation, Career, Disciplinary, Scholarship, and Doctoral Study codes, together with one CTU rector’s directive (SD 2015/02). Each PDF is split into overlapping character chunks (3000 characters per chunk, 300-character overlap), and each chunk is stored as one Weaviate object together with its title, source file, and an OpenAI `text-embedding-3-large` vector—the eight source documents yield 329 chunks in total. The OpenAI embedder was chosen for convenience and is interchangeable with a local model. Retrieval is hybrid (vector + BM25) with a configurable mix parameter $\alpha = 0.7$, returning the top- k chunks back to the language model as the tool result. Adding a new document is intentionally a single operation: drop the PDF (or plain-text file) into `voice-agent/src/data/FEL/` and restart the agent—the seeding routine in `voice-agent/src/live/rag.py` ingests every new file on the next boot, so extending Pepper’s institutional knowledge does not require touching any code.

A tool call from the language model goes through the LiveKit Agents framework: the

tool runs, and its return value is fed back into the model context as a tool-result message. Two operations run in the background: gesture dispatch (Section 4.7) does not block the agent on the animation finishing, and for tools that hit external HTTP endpoints a short filler utterance (“Let me check”) is played in parallel with the network call so the visitor hears immediate audio feedback while the lookup is in flight.

Every tool that can fail returns a structured payload with an **error** field and an **instruction** field telling the language model how to phrase the failure. For example `room-not-found` yields an instruction to ask the visitor to confirm the room code. The language model paraphrases the instruction in its next streamed sentence. There is no automatic fallback to a free language-model answer when an external service is down: the failure mode is always an explicit apology or follow-up question rather than a fabricated reply.

■ 4.7 Non-Verbal Behaviour

■ 4.7.1 Gestures

Pepper accompanies tool calls with a short body gesture. Each tool in the surface of Section 4.6.2 takes an extra **emotion** argument; when the language model calls the tool, the agent triggers the corresponding animation on Pepper without blocking on it.

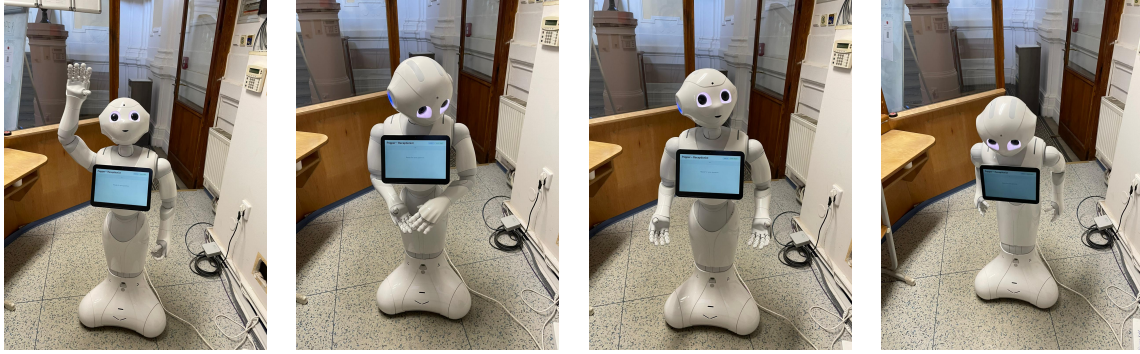
Fifteen emotion labels are exposed to the language model. They cover social openings and closings (greet, bow, goodbye), short reactions (affirm, deny, question), cognitive states the robot wants to show (think, don’t-know), explanatory dialogue (explain, emphasis, offer, address-user, calm), and a neutral fallback. Each label maps to a group of concrete animations, and the variant is picked at random on each call to avoid the obvious repetition that a single fixed animation per label would produce. Only silent animations are used so that canned vocalizations do not clash with the synthesized speech.

Because gestures fire on tool calls rather than on the streamed reply (Section 4.2.3), the gestures actually seen during a session lean toward the “thinking” and “looking up” part of the emotion vocabulary; the more expressive “explaining”, “offering”, or “emphasizing” gestures that the abandoned tool-only design could produce during the spoken reply itself are absent. This is a known trade-off in favour of streaming latency.

■ 4.7.2 Tablet

Pepper’s chest-mounted tablet is used as a secondary visual channel. During a session it shows a live transcript of the conversation—both what Pepper heard the visitor say and what Pepper said back—so the visitor can check that her question was understood correctly. When a tool returns concrete data such as a phone number, an office number, or a course code, that data stays on the tablet after Pepper has finished speaking, which gives the visitor time to read it or write it down.

The tablet state is owned by the **tablet-server** service, which renders the current view as HTML and pushes it to Pepper’s tablet through the robot **bridge**.



(a) greet

(b) thinking

(c) explain

(d) goodbye

Figure 4.4: Four representative gesture animations exposed to the language model.

4.8 Session Management

A receptionist session is the unit of interaction with a single visitor. The **voice-agent** worker for the currently scheduled condition is connected to the LiveKit room as a warm agent during idle: speech-processing models are already loaded, the LiveKit transport is established, and the agent is waiting for an activation signal. When a visitor greets Pepper, the session driver sends an activation signal over a LiveKit data channel; the agent greets the visitor, and dialogue begins. The session ends when the agent fires `end_conversation`, or after a silence timeout if the visitor does not respond. The warm-agent design is a deliberate choice driven by first-response latency: loading the speech-processing models takes several seconds on the target hardware, and keeping the worker alive across the idle state amortizes this cost.

4.8.1 Experimental Loop

For the experiment, sessions are dispatched in a loop on the Raspberry Pi. The loop runs one session at a time, alternates the assigned condition (Condition A and Condition B), and persists the next-up condition and participant identifier to a small JSON state file so the loop can resume after a restart.

4.9 Implementation and Deployment

All code is implemented in Python. The Raspberry Pi services run as Docker containers; the voice agents and the language-model server on the GPU server run as plain processes. Table 4.3 summarizes the principal third-party components used in the deployed system.

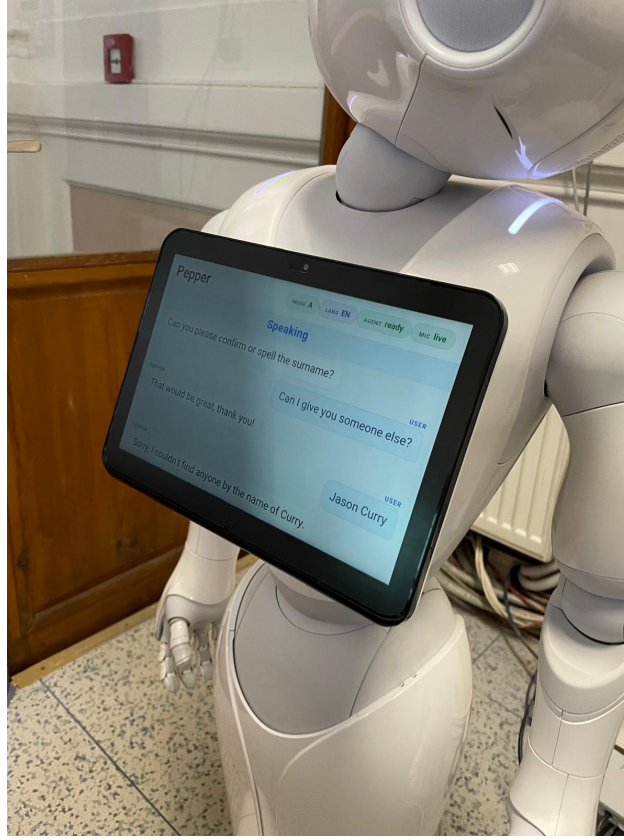


Figure 4.5: The live transcript helps users detect recognition errors. Here, the speech recognizer mistranscribed the surname of MSc. Jason Khoury as *Curry*.

Role	Component
WebRTC and room management	LiveKit
Acoustic echo cancellation	WebRTC AEC3 (Section 4.2.2)
Pepper audio playback	SSH + <code>paplay</code> into PulseAudio (Section 4.2.1)
Voice activity detection	Silero VAD
Speech-to-text (Condition A)	faster-whisper
Speech-to-text (Condition B)	OpenAI <code>gpt-4o-mini-transcribe</code>
Language model (Condition A)	Llama 3.1 8B Instruct (served by vLLM)
Language model (Condition B)	OpenAI <code>gpt-4o-mini</code>
Text-to-speech (Condition A)	Piper
Text-to-speech (Condition B)	OpenAI <code>gpt-4o-mini-tts</code>
Vector database (<code>query_search</code>)	Weaviate
Container orchestration	Docker Compose

Table 4.3: Principal third-party components of the deployed system.

Chapter 5

Experiments and Results

This chapter describes the user study that evaluates the receptionist system built in Chapter 4. The system was deployed at a real reception desk. Visitors interacted with it, and their experience was measured with a short questionnaire whose answers were linked to the recorded session log via a unique conversation identifier. The chapter is organized in five parts. Section 5.1 states the research questions and the experimental design. Section 5.2 describes the deployment site, the visitor flow, and how participants were recruited. Section 5.3 covers the questionnaire. Section 5.4 presents the quantitative results: latency and subjective ratings. Section 5.5 closes the chapter with the qualitative observations made by the experimenter during the deployment. Interpretation of these results, study limitations, and proposed remedies are taken up in Chapters 7 and 8.

5.1 Research Questions and Design

The study addresses two research questions:

- **RQ1.** How does an LLM-driven receptionist on the Pepper robot perform in a real institutional setting, both in terms of objective interaction quality (latency) and in terms of the subjective experience reported by visitors?
- **RQ2.** How do the two voice backends—Condition A (the locally hosted cascade) and Condition B (the cloud-hosted cascade)—compare in the same deployment and on the same task?

The study uses a **between-subjects** design, because an interaction with a receptionist is naturally a single visit. And the subjective experience of interacting with Pepper in one condition would likely influence the experience of interacting with her in the other condition. Each visitor interacts with the system once, in exactly one of the two conditions, and provides a single questionnaire response. The condition is assigned by the experimental loop wrapper (see Section 4.8.1), which alternates between Condition A and Condition B on each new session.

5.2 Procedure

5.2.1 Setting

The deployment site was the reception of Building E of FEE CTU at Prague. Pepper was placed behind the standard reception desk, facing visitors (mostly students and staff) who approach from the entrance corridor. A pane of glass separates the reception staff side, where Pepper stood, from the visitor side of the desk. With the microphone on Pepper's side, the glass and the distance made speech transcription unreliable. So external USB microphone described in Section 4.3.1 was mounted on the visitor side of the glass so that speech capture stayed close to the visitor. A short video of an interaction at the desk is available in the project repository [27].



Figure 5.1: Setting: the reception desk where the experiment was done.

5.2.2 Visitor Flow and Recruitment

The original plan was passive recruitment. A printed instructions board next to Pepper (Appendix A) was meant to be the only briefing: it explained what Pepper can help with, how to talk to her, and how to fill in the questionnaire afterwards. A visitor would read the board, speak to Pepper, and at the end of the session scan a QR code on Pepper's tablet that opened the questionnaire pre-filled with that session's participant identifier (a short tag of the form T07, which serves as the join key between the questionnaire row and the system log).

However, this did not work. Several visitors engaged with Pepper during the first hours of the deployment day, but none filled in the questionnaire. From that point onward the experimenter actively approached visitors near the reception, briefly explained what Pepper could help with, invited them to try her, and asked them to fill in the questionnaire afterwards. All 26 responses were collected on Monday 18 May 2026.

The switch turned out to carry an unexpected benefit: the experimenter was present for every interaction, both the ones that went well and the ones that fell apart. The systematic observations that came out of this presence are reported in Section 5.5.

5.2.3 Ethics and Data Handling

This research was approved by the Committee for Research Ethics at the Czech Technical University in Prague (reference number 0000-20/25/51902/EKČVUT). Visitors were informed about the study before they spoke to Pepper. The instructions board next to the desk stated that the conversation would be processed by the system and used for the thesis

evaluation (see Appendix A), and the experimenter additionally briefed each participant verbally during the active-recruitment phase (see Section 5.2.2). The questionnaire was anonymous—it asked for age, gender, and a short participant identifier, but no name—and the system logs do not contain visitor names either, so questionnaire rows and session logs are joined only through that short identifier. The session logs include the visitor’s microphone audio and the agent’s synthesized reply alongside the event stream; these, together with the questionnaire responses, are stored on the Raspberry Pi and in the project’s experiment-results folder.

The two backends differ in what leaves the institution. In Condition A, the visitor’s microphone audio, transcripts, and the agent’s replies stay on the Raspberry Pi and the departmental GPU server. In Condition B, every visitor utterance is sent to OpenAI for transcription, language-model inference, and speech synthesis, together with the tool-result text appended to the model context—which for the live-endpoint tools (e.g., `lookup_person`) includes the public faculty directory data returned by the lookup.

5.3 Questionnaire

The questionnaire was hosted on Google Forms. The full questionnaire is reproduced in Appendix B; this section describes its structure and the rationale for each block.

The questionnaire is organized into five blocks:

- **Identification and demographics**—four items: the participant identifier, self-reported gender, age, and familiarity with robots on a 5-point scale.
- **Custom Likert items**—three 5-point Likert items (“strongly disagree” to “strongly agree”) targeting whether the visitor got the information they were looking for, whether Pepper responded quickly enough, and whether her hand and arm gestures felt natural and fit what she was saying.
- **Categorical and open-ended items**—four items about the interaction content and the visitor’s perspective: what topic the visitor talked about, what problems occurred during the interaction, what the visitor would like Pepper to be able to help with in future, and whom they would prefer at the reception desk in general.
- **Godspeed scales**—twenty-one items from the Godspeed questionnaire [12], presented as bipolar adjective pairs on a 5-point scale, covering four of the five Godspeed subscales (Anthropomorphism, Animacy, Likeability, Perceived Intelligence). Godspeed was chosen because it is the most widely reported scale in the receptionist-robot literature surveyed in Chapter 2, which keeps the per-subscale means directly comparable to prior studies.
- **Open comment**—a final free-text item that invites the visitor to share anything else about their experience that the structured items did not capture.

The fifth Godspeed subscale, Perceived Safety, was *not* included. Its items measure the respondent’s self-reported emotional state as an indirect proxy for how safe the robot feels, which is most informative when there is a real possibility of physical contact with the robot.

5.4 Quantitative Results

The questionnaire collected 26 valid responses on a single deployment day. Twenty of those 26 responses could be matched back to a specific session log via the participant-identifier mechanism: 10 to a Condition A (local) session and 10 to a Condition B (cloud) session. The remaining 6 responses contained a participant identifier that did not uniquely map to a logged session, because the visitor scanned the printed backup QR after walking away and the on-tablet identifier did not get copied into the form correctly. These 6 responses are kept in the pooled descriptive statistics but are reported separately under “unknown” in the per-condition tables; they are excluded from the direct A–B comparison. The numbers and figures below were produced by the analysis scripts and notebooks in the project repository [27]; subsequent references give each script or notebook by its path relative to the repository root. The cleaned per-respondent responses, with each row matched to the corresponding session log, are kept at `docs/thesis/experiment/results/results.csv` in the same repository.

5.4.1 Sample

The 26 respondents were 22 men and 4 women, with ages ranging from 19 to 49 and a median of 25.5 years. Self-reported familiarity with robots, on a 5-point scale from “not familiar at all” (1) to “very familiar” (5), had a mean of 3.62 and a median of 4. This is the kind of sample one would expect at an engineering-school reception during teaching hours: mostly young men with some prior exposure to robots. The ratings reported below should be read with that audience in mind.

5.4.2 Objective Latency

The headline objective measurement is the time-to-first-audio (TTFA): the time from the end of the visitor’s utterance to the first audio frame submitted to Pepper’s audio device.

TTFA is computed per turn from each session’s event log, and each session is then summarized by the median TTFA across its turns. The two conditions are compared on these 10 per-session medians, so each visitor contributes one observation; the full computation is in `docs/thesis/experiment/evaluation/latency/latency_per_session.py` and `docs/thesis/experiment/evaluation/latency/latency_histogram.py`.

Table 5.1 reports the per-condition statistics. Condition A is descriptively faster: the median session is 1.39 s for the local cascade, compared with 3.30 s for the cloud cascade.

Condition	Sessions	Median (s)	Mean (s)
A (local)	10	1.39	1.31
B (cloud)	10	3.30	3.29

Table 5.1: Per-session time-to-first-audio (TTFA) for the two backends. Each session contributes its per-turn median; the table reports the median and mean across the 10 sessions per condition. The full per-turn distribution is shown in Fig. 5.2, and the tool vs no-tool comparison in Fig. 5.3.

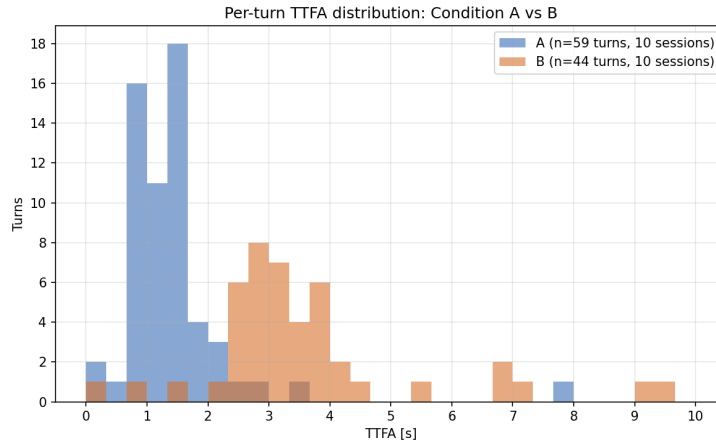


Figure 5.2: Per-turn TTFA distribution for the two conditions (59 turns from A and 44 turns from B, across 10 sessions per condition).

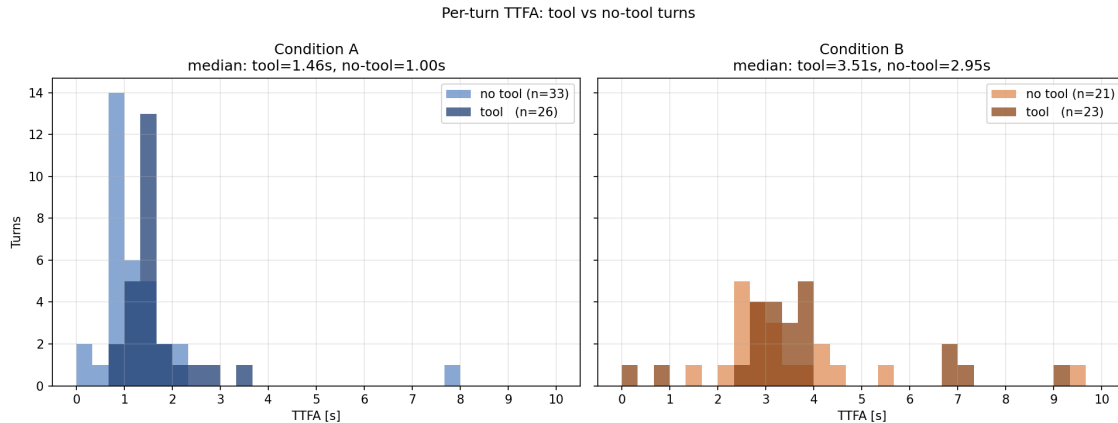


Figure 5.3: Per-turn TTFA split by whether the turn involved at least one tool call. Tool turns sit consistently to the right of no-tool turns. The extra latency is not from the HTTP round-trip—a short “Let me check” filler is played immediately on the tool call—but from the fact that tool calls are not streamed: the language model has to finish writing the full tool call before the tool can run (see Section 4.2.3).

5.4.3 Subjective Ratings

On every subjective item collected, respondents who experienced Condition B rated Pepper at least as highly as respondents who experienced Condition A.

Table 5.2 reports the per-condition means on the three custom Likert items. Condition B was rated higher than Condition A on all three.

The four Godspeed subscales follow the same direction. Table 5.3 reports the per-condition subscale means together with Cronbach’s α as a check of internal consistency. All four subscales reach $\alpha > 0.7$, which is the conventional threshold for acceptable reliability of a short subscale.

Item	Pooled	A (n=10)	B (n=10)	Unknown (n=6)
I got the information I was looking for	3.62	3.50	4.10	3.00
Pepper responded quickly	3.92	3.80	4.30	3.50
Pepper’s gestures felt natural and fit what she was saying	3.08	2.80	3.60	2.67

Table 5.2: Mean rating on the three custom Likert items (1–5, strongly disagree to strongly agree), split by condition. “Unknown” contains the six respondents whose questionnaire row could not be matched to a specific session log.

Subscale	α	Pooled	A (n=10)	B (n=10)	Unknown (n=6)
Anthropomorphism	0.72	2.72	2.58	2.84	2.73
Animacy	0.75	3.19	3.18	3.30	3.00
Likeability	0.81	4.33	4.20	4.46	4.33
Perceived Intelligence	0.80	3.45	3.38	3.58	3.33

Table 5.3: Godspeed subscale means (on a 1–5 scale) by condition, with Cronbach’s α on the pooled sample. “Unknown” contains the six respondents whose questionnaire row could not be matched to a specific session log.

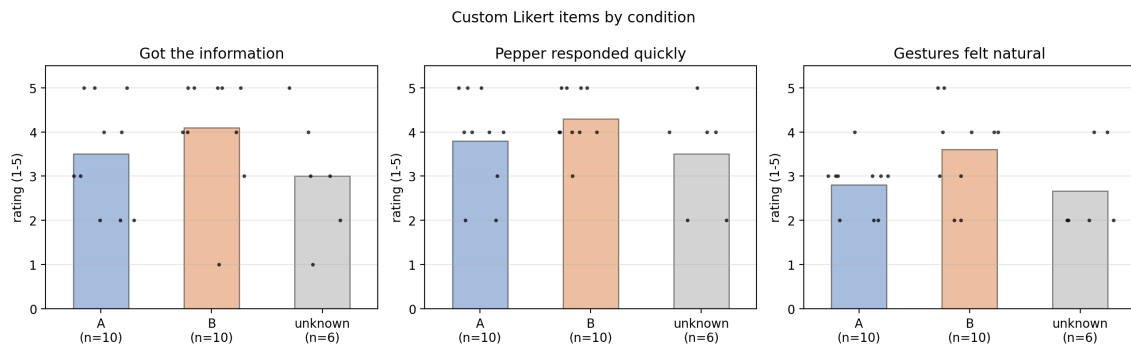


Figure 5.4: Mean rating on the three custom Likert items by condition. Bars show the condition mean; dots show individual respondents. “Unknown” covers respondents whose questionnaire row could not be matched to a session log.

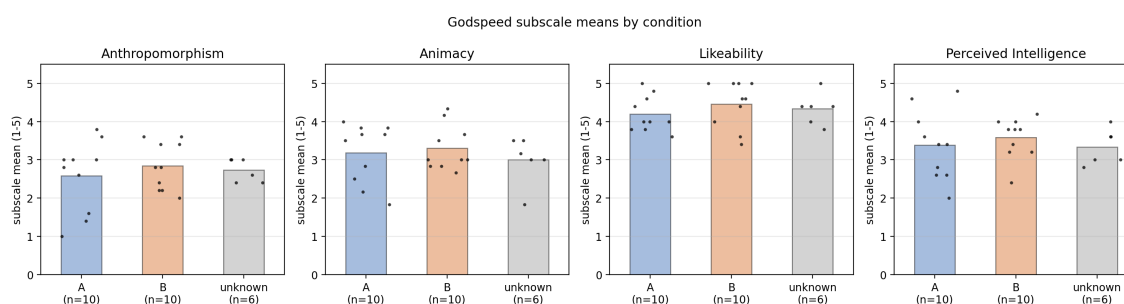


Figure 5.5: Godspeed subscale means by condition. Each subscale score is the mean of its items on the 5-point bipolar adjective scale. Bars show the condition mean; dots show the per-respondent subscale scores.

5.4.4 Visitor Preferences

One item asked who the visitor would prefer at the reception desk of Building E. Of the 26 respondents, 19 (73 %) preferred Pepper, 4 (15 %) preferred a human, and 3 (12 %) answered other. This number should be read with one site-specific context note: at the time of the deployment the human reception staff at Building E did not regularly speak English. For visitors who do not speak Czech, Pepper was therefore not a robot-versus-human comparison but the only English-speaking option at the desk, which plausibly inflates the Pepper preference within that subset.

5.5 Observations from Experiment

Because the experimenter was present for every session, the following recurring issues were noted during the day. They applied to both conditions equally unless noted otherwise.

- **Pepper’s voice behind the glass.** The glass partition attenuated Pepper’s voice on the visitor side. Turning the volume up to compensate produced audible echo, so several visitors ended up reading Pepper’s words on the live tablet transcript rather than relying on the audio alone.
- **Background chatter.** The reception is located in a busy corridor. The microphone occasionally picked up nearby conversation as if it were the visitor speaking, and Pepper would politely reply to it.
- **Repeated filler on tool retries.** A short “Let me check” utterance is played in parallel with each tool call to cover the silence before Pepper’s reply. When a tool failed and Pepper retried with a slightly different argument, the same filler line was spoken on every retry—in one case three times in a row—which sounded strongly unnatural. Limiting the filler to fire at most once per visitor turn would be enough to prevent this.
- **Guided versus self-directed visitors.** Visitors who were briefed by the experimenter on what Pepper can help with usually had a smooth conversation. Visitors

who approached cold tended to ask for things outside Pepper's capabilities, and the conversation degraded into apologies.

Chapter 6

Conclusion

This thesis presents the design and evaluation of an LLM-driven receptionist robot for social interaction. The implemented system runs on a Pepper robot. A cascaded speech-to-text, large language model, and text-to-speech pipeline handles the conversation, a small fixed set of helper functions covers the questions a faculty visitor is likely to ask (room directions, staff lookup, lecture schedule, canteen menu), and short body gestures together with a live tablet transcript accompany every spoken reply. The same agent code supports two interchangeable backends: a fully local cascade built around **faster-whisper**, Llama 3.1 8B Instruct served by vLLM, and a Piper voice; and a cloud cascade built around OpenAI’s **gpt-4o-mini** family of models. The design also keeps the system easy to extend: a new capability is added by writing a single tool and naming it in the system prompt, Pepper’s persona can be changed by editing the prompt itself, and new institutional knowledge can be added by dropping a PDF into the seed folder.

The system was deployed at the real reception desk of Building E at FEE CTU for one day, alternating between the two backends across visitors. Twenty-six visitors completed a short questionnaire combining four Godspeed subscales and three custom Likert items, and a closing categorical question on whom they would prefer at the desk. The headline finding is a two-sided trade-off. Condition B (cloud) was rated at least as highly as Condition A (local) on every subjective item, with gaps of 0.1 to 0.3 points on the Godspeed subscales and up to 0.8 on the gesture item. Condition A was descriptively about twice as fast on the median session (1.39 s vs 3.30 s session-median time-to-first-audio across 10 sessions per condition) and does not incur per-token inference fees once the hardware is available, against the cloud cascade’s metered USD 0.29 per active conversation hour. Of the 26 respondents, 19 (73 %) preferred Pepper at the reception over a human.

The thesis delivers the full LLM-driven receptionist system specified by the assignment (Section 1.2), including the side-by-side comparison of a local and a cloud voice cascade on the same task. Because the two conditions differ in speech recognition, language model, and speech synthesis simultaneously, the comparison is best read as a comparison of complete cascades rather than as an isolated test of the language model alone (see Section 7.1). Interpretation of the local-vs-cloud trade-off and the limitations of the study are discussed in Chapter 7; concrete next steps surfaced by the deployment are listed in Chapter 8.

Chapter 7

Discussion

7.1 Limitations

The study has several limitations that constrain how strongly the results in Chapter 5 can be read.

- **The comparison is not an isolated LLM test.** Condition A and Condition B differ in three components at once—speech-to-text, language model, and text-to-speech—so any subjective difference between the two conditions cannot be attributed to the language model alone. The cloud cascade pairs `gpt-4o-mini` with the `nova` voice and `gpt-4o-mini-transcribe`, both of which are plausibly better than the local Piper voice and `faster-whisper` on this audience. The results should therefore be read as a comparison of two complete local and cloud voice cascades on the same task, rather than as an isolated comparison of the language models.
- **Sample size and condition assignment.** Of the 26 questionnaire responses, 20 could be matched back to a specific session, 10 per condition (the remaining 6 carried no participant identifier). Ten respondents per condition is too small to generalize beyond this study, so the per-condition tables in Chapter 5 are reported as observations from this deployment rather than broader claims. The condition was alternated rather than fully randomised across visitors (see Section 4.8.1), so a residual confound with time-of-day or visitor population on the deployment day cannot be ruled out.
- **Self-selection.** Visitors who chose to engage with the robot are likely more positively disposed toward robots than the average passer-by. This is expected to inflate subjective ratings across both conditions and limits how far the absolute scores can be generalized.
- **English-only language bias.** The system speaks English only, but the deployment site is a Czech university. Visitors who do not speak English comfortably either skipped the interaction altogether or rated the experience under the additional load of working in a second language. The sample therefore over-represents English speakers, and the ratings reflect English-language interaction quality rather than the experience a Czech-speaking visitor would have. Multilingual support is listed as future work (Section 8.2).
- **Recruitment switch.** Passive recruitment did not produce questionnaire responses, so the experimenter actively approached visitors and briefly explained the study (see Section 5.2.2). This likely improved task discoverability—the guided-versus-self-directed

observation in Section 5.5 shows that briefed visitors had smoother conversations—but it also primed the visitors’ expectations of what Pepper can do and shifted the kinds of questions they asked.

- **Retrieval quality was not separately evaluated.** The `query_search` tool was rarely invoked in the field deployment (Section 4.6.2) because visitor questions almost all landed on the four live-endpoint tools. The retrieval-augmented question answering called for in the thesis assignment is therefore present as a working component but not benchmarked in isolation; a small offline evaluation over the seeded document corpus—example questions, the retrieved chunks, and whether the answer was correct—would close this gap and remains a natural follow-up item.
- **Gesture evaluation is rough.** The custom Likert item asks whether Pepper’s gestures *felt natural and fit what she was saying*, which is a perceived-appropriateness rating, not a detailed measure of how well each gesture’s meaning matched the spoken text. Because gestures fire on tool calls rather than on the streamed reply (Section 4.2.3), the visible vocabulary is also dominated by “thinking” and “looking up” gestures, so the item should be read as evidence about that subset of the design rather than about the full gesture surface.

7.2 Implications

The two backends pull in opposite directions on every axis that matters for a real deployment. Condition B was perceived as more pleasant to interact with; Condition A was the quicker one to respond, and the only one that keeps the visitor’s audio on local hardware. The rest of this section discusses each of these in detail.

- **Why B was slightly preferred**—On general-knowledge and reasoning benchmarks, `gpt-4o-mini` sits noticeably above Llama 3.1 8B Instruct (about 13 points on MMLU and 15 points on HumanEval [33, 39], around 100 Elo points in the Chatbot Arena), and the cloud cascade also pairs it with a smoother synthesized voice (`nova`) and a more accurate speech-to-text model. A more capable language model with a warmer voice and cleaner transcripts is expected to be rated higher, and on this small sample that is what happens: the gap is consistent in direction (0.1–0.3 points on the Godspeed subscales, up to 0.8 on the gesture item) but small in size. It is best read as a tendency rather than a proven difference.
- **The latency paradox**—Condition A was descriptively about twice as fast as Condition B on the median session, yet Condition B was rated slightly higher on the speed item (4.30 vs 3.80). There are three plausible reasons, not mutually exclusive. First, both medians (1.39 s and 3.30 s) fall inside a range that feels conversational, so neither is heard as a delay. Second, the two cascades differ in speech-to-text and text-to-speech components as well as in the language model (see the LLM-isolation limitation in Section 7.1); the cloud cascade’s smoother voice and more accurate transcripts plausibly shape *perceived* speed independently of the measured first-audio time. Third, plain sampling noise: with ten respondents per condition, a 0.5-point gap

could easily come from random variation alone. Either way the practical message is the same—at this scale, raw measured latency was not what made Pepper feel fast or slow.

- **Cost**—The two backends have opposite cost shapes.

The cloud cascade is pay-as-you-go: every minute of conversation is billed against the OpenAI API, and an hour with no visitors costs nothing. At prices accessed on 18 May 2026 [40], the ten Condition B sessions in this study corresponded to approximately USD 0.29 per active conversation hour. The bill was computed by `docs/thesis/experiment/evaluation/cost/compute_b_costs.py` against the official OpenAI usage export. OpenAI pricing is model-dependent and changes over time, so this estimate should be interpreted as a snapshot rather than a stable long-term price.

The local cascade has no per-token API fee, but it is not free: it requires up-front hardware, plus ongoing electricity, maintenance, administrator time, GPU opportunity cost, and the effort of pulling in model updates. The remainder of this paragraph focuses on the hardware up-front cost, which dominates at the usage scale of a faculty reception; the other items are smaller and harder to attribute and are left out of the comparison. The GPU used here (see Section 4.3.2) lists for about USD 1,250 [31]. Dividing the GPU price by the cloud per-hour rate gives a break-even of roughly 4,170 hours of active conversation. Depending on how much the robot is actively used—between two and six hours of conversation per day is a reasonable range for a faculty reception—the GPU pays for itself in roughly two to six years.

- **Upgradability**—Switching the cloud cascade to a different language model is a one-line configuration change: the `model` string can be swapped for any other OpenAI-hosted model, or the LLM component can be replaced with a plugin for a different provider. This makes it cheap to follow frontier improvements.

Upgrading the local cascade is more involved. Moving to a 32B or 70B language model means either accepting that it does not fit on the existing GPU alongside the speech components or buying a larger GPU. The local backend is therefore conservative by construction: it follows the slow tail of open-weight improvements at the size class it can fit.

- **Data sovereignty**—The cost and upgradability advantages of the cloud cascade come with one non-negotiable downside: every visitor utterance, transcript, and reply leaves the deployment site and is processed on the provider’s infrastructure. Whether that is acceptable is a deployment decision and depends on the institution. The local cascade keeps speech recognition, dialogue generation, and speech synthesis on hardware controlled by the operator; the one current exception is that `query_search` computes embeddings through the OpenAI API at ingestion and query time as a convenience choice, so RAG use is not strictly local under Condition A. The embedder is interchangeable and swapping it for a local model is a one-component change (see Section 5.2.3).
- **Picking a backend**—The choice between the two backends depends on the needs of the deployment. The trade-offs above—subjective quality, latency, cost shape, upgrade

path, and where the audio is processed—pull in different directions, and which of them matters most is specific to the application.

Chapter 8

Future Work

The deployment in Chapter 5 and the field observations in Section 5.5 surfaced a number of concrete follow-up items. They are grouped here into two categories: small implementation fixes for issues seen in the field, and larger directions that would meaningfully extend the work.

8.1 Implementation Fixes Surfaced by the Field Study

- **Make visitor-side capabilities more discoverable.** Visitors who approached Pepper without a verbal briefing tended to ask for things outside the tool surface and ended in apologies (Section 5.5). Two concrete remedies, in increasing engineering effort: a richer tablet display that shows the available capabilities as quickly-readable chips while Pepper is idle; and a short spoken self-introduction by Pepper on visitor approach (“I can help you find a room, a person, a course, or the canteen menu”), triggered by a simple presence cue.
- **Better microphone capture.** A single omnidirectional microphone on the visitor’s side of the glass occasionally forwarded background corridor chatter into the pipeline (Section 5.5). This could be solved, for example, by a push-to-talk button on the visitor’s side of the desk so that the microphone is gated on only while a visitor is actively speaking to Pepper, which would also prevent Pepper from picking up unrelated conversation during the rest of the day. A directional microphone pointed at the visitor would address the same issue acoustically by picking up less sound from the corridor.

8.2 Larger Directions

- **Multilingual support, especially Czech.** The current system speaks English only. Switching to a multilingual speech-to-text model and a multilingual text-to-speech voice, and prompting the language model to detect and follow the visitor’s language, would broaden the audience significantly.
- **Memory of returning visitors.** The current agent has no persistent memory across sessions. Lightweight personalization—recognizing a returning face or accepting a name once and remembering it across visits—would be a natural extension and would interact with the previous item.

- **Unattended all-day operation.** The deployment described in Chapter 5 ran for one day with the experimenter present. A permanent installation would need to stand on its own: starting in the morning, recovering by itself from a stuck session or a network blip, sleeping when no one is around to save power, and re-initialising the agent on idle so the next visitor’s first turn is not affected by accumulated state from the previous conversation.
- **Vision input for richer context.** The current agent is blind: it knows nothing about the visitor’s appearance, where they are looking, or what is happening around the reception desk. A natural extension is to add an additional tool, for example `look_around`, that captures a still image from one of Pepper’s cameras and sends it to a small vision-language model running alongside the local cascade; the model returns a short textual description of the scene that is fed back to the agent as the tool result. The agent could call this tool when context is unclear (“is the visitor still there?”, “does the visitor look confused?”, “is there a queue building up behind them?”), without changing anything else in the pipeline. Both points come with trade-offs: each vision call adds image capture and model inference on top of the normal turn latency, and processing visitor faces and the surroundings raises stronger privacy concerns than the audio-only setup of the current system.



Bibliography

- [1] R. Gockley, A. Bruce, J. Forlizzi, M. Michalowski, A. Mundell, S. Rosenthal, B. Sellner, R. Simmons, K. Snipes, A. C. Schultz, and J. Wang, “Designing robots for long-term social interaction,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2005, pp. 1338–1343.
- [2] T. Kanda, M. Shiomi, Z. Miyashita, H. Ishiguro, and N. Hagita, “A communication robot in a shopping mall,” *IEEE Transactions on Robotics*, vol. 26, no. 5, pp. 897–913, 2010.
- [3] X. Chen, K. Luo, T. Gee, and M. Nejati, “Does ChatGPT and Whisper make humanoid robots more relatable?” arXiv:2402.07095, 2024.
- [4] A. Radford *et al.*, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 28 492–28 518.
- [5] D. Herath, J. B. Grant, A. Rodriguez, and J. L. Davis, “First impressions of a humanoid social robot with natural language capabilities,” *Scientific Reports*, vol. 15, p. 19715, 2025.
- [6] J. Qu, M. Jarosz, and B. Sniezynski, “Robot control platform for multimodal interactions with humans based on ChatGPT,” *Applied Sciences*, vol. 14, no. 17, p. 8011, 2024.
- [7] N. Cherakara, F. Varghese, S. Shabana, N. Nelson, A. Karukayil, R. Kulothungan, M. A. Farhan, B. Nettet, M. Moujahid, T. Dinkar, V. Rieser, and O. Lemon, “FurChat: An embodied conversational agent using LLMs, combining open and closed-domain dialogue with facial expressions,” in *Proceedings of the 24th Annual Meeting of the Special Interest Group on Discourse and Dialogue (SIGDIAL)*, 2023, arXiv:2308.15214.
- [8] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, vol. 205, 2022, pp. 1769–1782.
- [9] M. Ahn, A. Brohan, N. Brown *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.

- [10] S. Saunderson and G. Nejat, “How robots influence humans: A survey of nonverbal communication in social human-robot interaction,” *International Journal of Social Robotics*, vol. 11, 2019.
- [11] M. Salem, F. Eyssel, K. Rohlfing, S. Kopp, and F. Joublin, “To err is human(-like): Effects of robot gesture on perceived anthropomorphism and likability,” *International Journal of Social Robotics*, vol. 5, no. 3, 2013.
- [12] C. Bartneck, D. Kulić, E. Croft, and S. Zoghbi, “Measurement instruments for the anthropomorphism, animacy, likeability, perceived intelligence, and perceived safety of robots,” *International Journal of Social Robotics*, vol. 1, no. 1, pp. 71–81, 2009.
- [13] C. M. Carpinella, A. B. Wyman, M. A. Perez, and S. J. Stroessner, “The robotic social attributes scale (RoSAS): Development and validation,” in *ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, 2017, pp. 254–262.
- [14] M. Heerink, B. Kröse, V. Evers, and B. Wielinga, “Assessing acceptance of assistive social agent technology by older adults: the Almere model,” *International Journal of Social Robotics*, vol. 2, no. 4, pp. 361–375, 2010.
- [15] A. Vaswani *et al.*, “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.
- [16] OpenAI, “OpenAI tokenizer,” <https://platform.openai.com/tokenizer>, 2026, accessed: 2026-03-03.
- [17] J. Wei *et al.*, “Emergent abilities of large language models,” *Transactions on Machine Learning Research*, 2022.
- [18] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, 2020.
- [19] Z. Ji *et al.*, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, 2023.
- [20] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, 2022.
- [21] H. W. Chung *et al.*, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, 2022.
- [22] T. B. Brown *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, 2020.
- [23] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, 2022.
- [24] OpenAI, “Function calling guide,” <https://developers.openai.com/api/docs/guides/function-calling/>, 2024, accessed: 2026-03-03.
- [25] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- [26] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” *International Conference on Learning Representations*, 2023.
- [27] L. Navara, “LLM-Driven Receptionist Robot for Social Interaction — source code,” [Online]. Available: <https://github.com/navarlu/Pepper/tree/thesis-v1.2>, 2026, version: tag `thesis-v1.2`.
- [28] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560 (later released as the Letta agent framework), 2023.
- [29] A. K. Pandey and R. Gelin, “A mass-produced sociable humanoid robot: Pepper, the first machine of its kind,” *IEEE Robotics & Automation Magazine*, vol. 25, no. 3, pp. 40–48, 2018.
- [30] SoftBank Robotics, “NAOqi framework documentation,” http://doc.aldebaran.com/2-5/index_dev_guide.html, 2018, accessed May 2026.
- [31] NVIDIA Corporation, “NVIDIA RTX 4000 SFF Ada Generation,” <https://www.nvidia.com/en-us/products/workstations/rtx-4000-sff/>, 2026, manufacturer product page, accessed May 2026.
- [32] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *ACM Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [33] A. Grattafiori *et al.*, “The Llama 3 herd of models,” arXiv:2407.21783, 2024.
- [34] LiveKit, “LiveKit documentation,” <https://docs.livekit.io/>, 2026, accessed May 2026.
- [35] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: a speech-text foundation model for real-time dialogue,” arXiv:2410.00037, 2024.
- [36] Silero Team, “Silero VAD: pre-trained enterprise-grade Voice Activity Detector,” <https://github.com/snakers4/silero-vad>, 2024, accessed May 2026.
- [37] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, “The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models,” in *International Conference on Machine Learning (ICML)*, 2025.
- [38] L. Navara, K. Zanáška, F. Žemlička, and K. Vylínek, “FELChat: Retrieval-augmented chatbot over CTU/FEE regulations,” GitHub repository, branch `localhost_setup2`, 2025, https://github.com/navarlu/FELChat/tree/localhost_setup2.
- [39] OpenAI, “GPT-4o mini: Advancing cost-efficient intelligence,” <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, 2024.
- [40] —, “OpenAI API Pricing,” <https://openai.com/api/pricing/>, 2026, token pricing for `gpt-4o-mini`, `gpt-4o-mini-transcribe`, and `gpt-4o-mini-tts`, accessed May 2026.



Appendix A

Visitor Instruction Board

This appendix reproduces the printed instruction board placed next to Pepper at the reception desk during the experiment. The board was the only briefing visitors received under the original passive-recruitment plan (Section 5.2.2) and explained what Pepper can help with, how to talk to her, and how to fill in the post-interaction questionnaire.

Meet Pepper - The New Receptionist

Part of a research experiment for the diploma thesis: *Design of an LLM-Driven Receptionist Robot for Social Interaction*

Informed Consent — Please Read

By speaking into the microphone, you agree that your **audio recording may be stored and used** for this thesis research and its evaluation.

If you do not consent, please do not speak to Pepper.

What Pepper Can Help You With

- **Navigation** — directions to rooms in the building (please use the **room code**, e.g. E-107)
- **Lecture schedule** — finding the time and place of a lecture (please use the **subject code**, e.g. HRO)
- **Staff contacts** — finding contact details for faculty members and staff (*note: Czech names may sometimes be misrecognised — try spelling or repeating*)
- **Mensa menu** — today's menu in the canteen
- **General conversation** — feel free to just chat with her

Language: Pepper currently speaks **English only**.

How to Talk to Her

1. **Watch the tablet** — it shows Pepper's current state
2. **When she is ready**, just greet her to start the conversation (e.g. "Hi Pepper")
3. **Talk naturally** — you can **interrupt her** at any time if needed
4. **End the conversation** by saying goodbye in your own words — the tablet will then show a **QR code with your pre-filled conversation ID** for the questionnaire
5. **Missed the QR code?** No problem — scan the code on the board instead and enter your **conversation ID** manually

After Your Interaction

It would help my research enormously if you could fill in a [short 3-minute questionnaire](#) about your experience — the results will be used to evaluate the interaction quality.

 **Before you leave:** make sure you have your [conversation ID](#) from Pepper's tablet — the questionnaire requires it to link your answers to the interaction.



Contact

Something not working? Questions about the research? [Lukas Navara](#) —

Thank you for taking part!



Appendix B

Post-Interaction Questionnaire

This appendix reproduces the full post-interaction questionnaire used in the experiment (Section 5.3). The questionnaire was hosted on Google Forms during the deployment; the layout below is exported from that form.

Pepper — Feedback + Godspeed 1–4

3 min questionnaire for feedback on interaction with Pepper.

This experiment is part of my diploma thesis on topic of:

Design of an LLM-Driven Receptionist Robot for Social Interaction.

Your responses will be stored anonymously and used only for the purposes of this thesis.

Thank you for participating.

* Indicates required question

Conversation ID: *

(shown on Pepper's tablet during your conversation)

Gender: *

- ☐ Male
- ☐ Female
- ☐ Prefer not to say

Age: *

How familiar are you with robots? *

Not familiar at all 1 2 3 4 5 Very familiar

I got the information I was looking for. *

Strongly disagree 1 2 3 4 5 Strongly agree

Pepper responded to my questions quickly. *

Strongly disagree 1 2 3 4 5 Strongly agree

Pepper's hand and arm gestures felt natural and fit what she was saying. *

Strongly disagree 1 2 3 4 5 Strongly agree

What did you talk with Pepper about? *

- ☐ General conversation
- ☐ Subject schedule
- ☐ Contact information
- ☐ Mensa menu
- ☐ Navigate to some room
- ☐ Other: _____

Was there any problem during the interaction? *

- ☐ No problem
- ☐ Pepper was slow to respond
- ☐ Pepper did not respond at all
- ☐ The provided information was false
- ☐ It was hard to understand what was Pepper saying
- ☐ It was hard to see the transcription on the tablet
- ☐ The transcription of what I said was incorrect
- ☐ Other: _____

What would you like Pepper to be able to help you with?

Who would you prefer here at reception E? *

- ☐ Human
- ☐ Pepper
- ☐ No one
- ☐ Other: _____

How did Pepper seem to you?

Fake — Natural *

Fake 1 2 3 4 5 Natural

Machinelike — Humanlike *

Machinelike 1 2 3 4 5 Humanlike

Unconscious — Conscious *

Unconscious 1 2 3 4 5 Conscious

Artificial — Lifelike *

Artificial 1 2 3 4 5 Lifelike

Moving rigidly — Moving elegantly *

Moving rigidly 1 2 3 4 5 Moving elegantly

Dead — Alive *

Dead 1 2 3 4 5 Alive

Stagnant — Lively *

Stagnant 1 2 3 4 5 Lively

Mechanical — Organic *

Mechanical 1 2 3 4 5 Organic

Artificial — Lifelike *

Artificial 1 2 3 4 5 Lifelike

Inert — Interactive *

Inert 1 2 3 4 5 Interactive

Apathetic — Responsive *

Apathetic 1 2 3 4 5 Responsive

Dislike — Like *

Dislike 1 2 3 4 5 Like

Unfriendly — Friendly *

Unfriendly 1 2 3 4 5 Friendly

Unkind — Kind *

Unkind 1 2 3 4 5 Kind

Unpleasant — Pleasant *

Unpleasant 1 2 3 4 5 Pleasant

Awful — Nice *

Awful 1 2 3 4 5 Nice

Incompetent — Competent *

Incompetent 1 2 3 4 5 Competent

Ignorant — Knowledgeable *

Ignorant 1 2 3 4 5 Knowledgeable

☐ ☐ ☐ ☐ ☐

Irresponsible — Responsible *

Irresponsible 1 2 3 4 5 Responsible

☐ ☐ ☐ ☐ ☐

Unintelligent — Intelligent *

Unintelligent 1 2 3 4 5 Intelligent

☐ ☐ ☐ ☐ ☐

Foolish — Sensible *

Foolish 1 2 3 4 5 Sensible

☐ ☐ ☐ ☐ ☐

Anything else you'd like to share?

Feedback, suggestions, or anything not covered by the questions above.

Thank you. Bye!